Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_08_)

Implementation of Program Counter and Instruction Memory

Student Name: ___Areeb Fatima_____ Roll No: ____24I-6543_____ Section: _B_

Date performed: ___14th October____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this first task, I basically make a 4-bit working program counter functionalities with instruction memory along with test bench for simulating and verifying the results. In PC, I took clk, reset, address and PC_out and when reset is zero then PC_out is address, else zero output and then in IM, I put PC_out output in address of IM and then Instruction in that address and assign the memory location to different instructions and in test bench, I verified the output simultaneously.

Code:

```
module program_counter( clk, reset, addr, pc_out);

input clk, reset;

input [3:0] addr;

output reg [3:0] pc_out;

always @ (posedge clk or posedge reset)

begin

   if(reset)

     pc_out <= 4'b0000;

   else

    pc_out <= addr;

end

endmodule

module instruction_memory(addr, instruction);

input [3:0] addr;

output reg [3:0] instruction;

reg [3:0] memory [0:15];

initial

begin
```

```
memory[0]= 4'b0000;

memory[1]= 4'b0001;

memory[2]= 4'b0010;

memory[3]= 4'b0011;

memory[4]= 4'b0100;

memory[5]= 4'b0101;

memory[6]= 4'b0110;

memory[7]= 4'b0111;

memory[8]= 4'b1000;

memory[9]= 4'b1001;

memory[10]= 4'b1010;

memory[11]= 4'b1011;

memory[12]= 4'b1100;

memory[13]= 4'b1101;

memory[14]= 4'b1110;

memory[15]= 4'b1111;

end

always @ (addr)

begin

instruction= memory[addr];

end

endmodule

module tb_program_counter();
```

```
reg clk, reset;

reg [3:0] addr;

wire [3:0] pc_out;

wire [3:0] instruction;

program_counter pc(.clk(clk), .reset(reset), .addr(addr), .pc_out(pc_out));

instruction_memory im(.addr(pc_out), .instruction(instruction));

always begin

#5 clk= ~clk;

end

initial

begin

clk=0;

reset=0;

addr=0;

reset=1;

#10 reset=0;

#10 addr=1;

#10 addr=2;

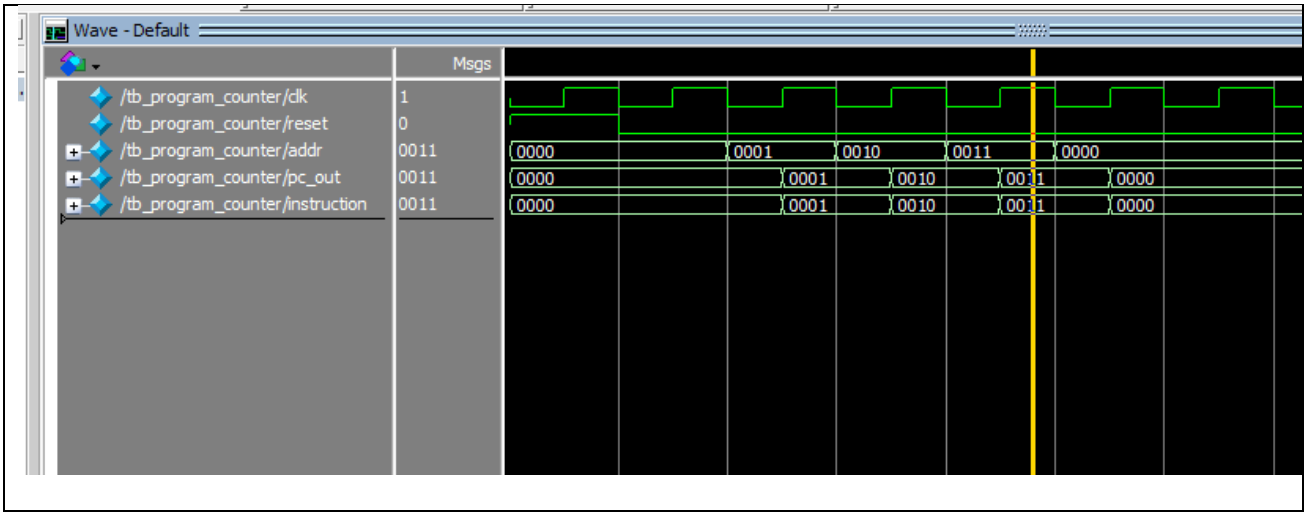
#10 addr=3;

#10 addr=0;

end

endmodule
```

Output:



Task 2:

Description:
In this 2nd task, I basically make a 8-bit working program counter with instruction memory along with test bench for simulating and verifying the results. In PC, I took clk, reset and PC_out and when reset is zero then PC_out is incremented to 1, else zero output and then in IM, I put PC_out output in address of IM and then Instruction in that address and put output 0 if address > 16 and assign the memory location to different instructions and in test bench, I verified the output simultaneously.
Code:

```
module program_counter( clk, reset, pc_out);

input clk, reset;

output reg [3:0] pc_out;

always @ (posedge clk or posedge reset)

begin

   if(reset)

     pc_out <= 4'b0000;

   else

    pc_out <= pc_out + 1'b1;
```

```
end

endmodule

module instruction_memory(addr, instruction);

input [3:0] addr;

output reg [7:0] instruction;

reg [7:0] memory [0:15];

initial

begin

memory[0]= 8'b00000000;

memory[1]= 8'b00000001;

memory[2]= 8'b00000010;

memory[3]= 8'b00000011;

memory[4]= 8'b00000100;

memory[5]= 8'b00000101;

memory[6]= 8'b00000110;

memory[7]= 8'b00000111;

memory[8]= 8'b00001000;

memory[9]= 8'b00001001;

memory[10]= 8'b00001010;

memory[11]= 8'b00001011;

memory[12]= 8'b00001100;

memory[13]= 8'b00001101;

memory[14]= 8'b00001110;
```

```
memory[15]= 8'b00001111;

end

always @ (addr)

begin

   if ( addr < 16)

     instruction= memory[addr];

   else

     instruction= 8'b00000000;

end

endmodule

module tb_program_counter();

reg clk, reset;

wire [3:0] pc_out;

wire [7:0] instruction;

program_counter pc(.clk(clk), .reset(reset), .pc_out(pc_out));

instruction_memory im(.addr(pc_out), .instruction(instruction));

always begin

#5 clk= ~clk;

end

initial

begin

clk=0;

reset=0;
```

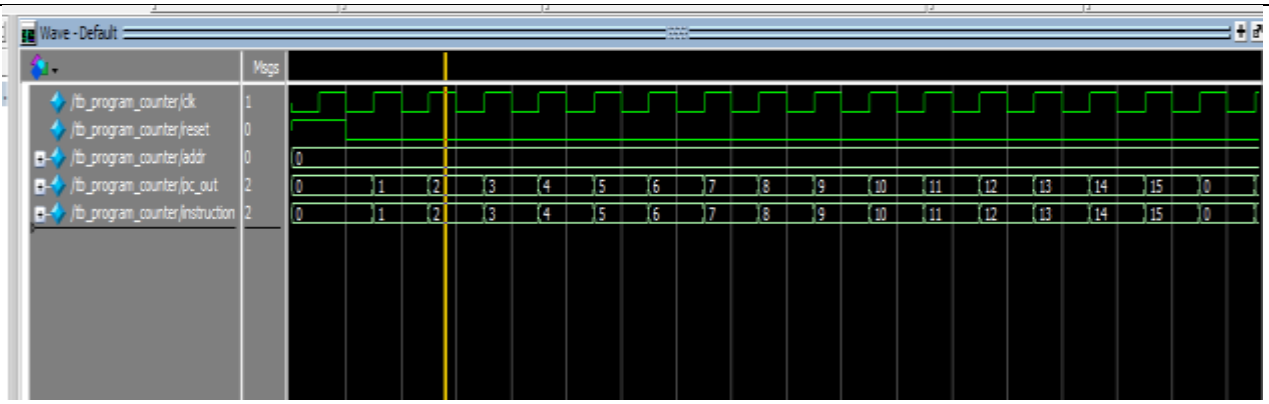
```
reset=1;

#10 reset=0;

end

endmodule
```

Output:



Task 3:

Description:

In this 3rd task, I basically make a 32-bit working program counter with instruction memory along with test bench for verifying the results. In PC, I took clk, reset, truncated address and PC_out and when reset is zero then PC_out is incremented to 4, else zero output and truncate the lower bits for indexing and then in IM, I put truncated address in address of IM and then Instruction in that address and assign the memory location to different instructions and in test bench, I verified the output simultaneously.

Code:

```
module program_counter( clk, reset, truncated_addr, pc_out);

input clk, reset;

output [3:0] truncated_addr;

output reg [31:0] pc_out;

always @ (posedge clk or posedge reset)

begin
```

```
    if(reset)
      pc_out <= 32'h00000000;
    else
     pc_out <= pc_out + 32'd4;
end
assign truncated_addr= pc_out[5:2]; //lower bits truncation
endmodule
module instruction_memory(addr, instruction);
input [3:0] addr;
output reg [31:0] instruction;
reg [31:0] memory [0:15];
initial
begin
memory[0]= 32'h00000000;
memory[1]= 32'h00000001;
memory[2]= 32'h00000010;
memory[3]= 32'h00000011;
memory[4]= 32'h00000100;
memory[5]= 32'h00000101;
memory[6]= 32'h00000110;
memory[7]= 32'h00000111;
memory[8]= 32'h00001000;
memory[9]= 32'h00001001;
```

```
memory[10]= 32'h00001010;

memory[11]= 32'h00001011;

memory[12]= 32'h00001100;

memory[13]= 32'h00001101;

memory[14]= 32'h00001110;

memory[15]= 32'h00001111;

end

always @ (*)

begin

   if ( addr < 16)

     instruction= memory[addr];

   else

     instruction= 32'h00000000;

end

endmodule

module tb_program_counter();

reg clk, reset;

wire [3:0] truncated_addr;

wire [31:0] pc_out;

wire [31:0] instruction;

program_counter pc(.clk(clk), .reset(reset), .truncated_addr(truncated_addr), .pc_out(pc_out));

instruction_memory im(.addr(truncated_addr), .instruction(instruction));

always begin
```

```
#5 clk= ~clk;

end

initial

begin

clk=0;

reset=0;

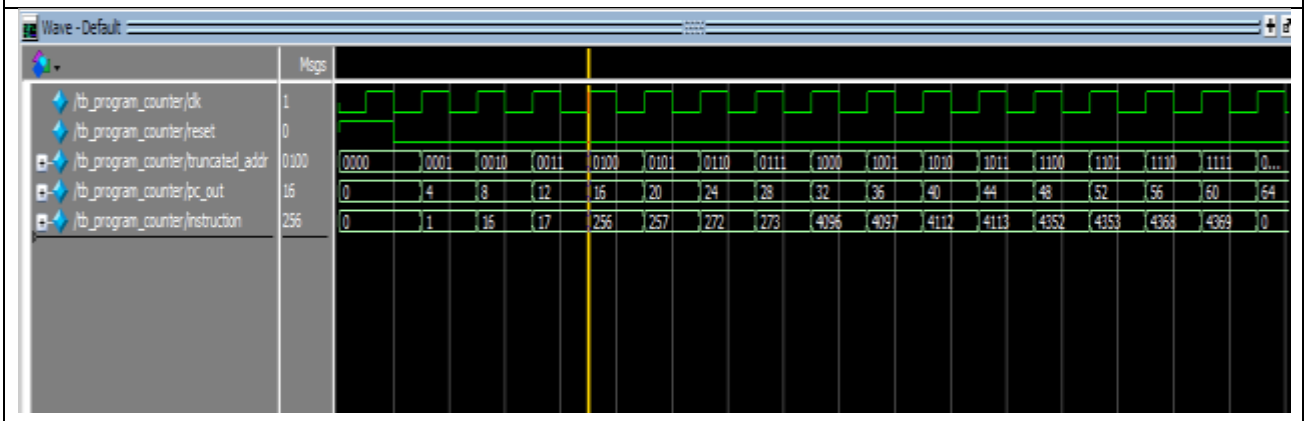
reset=1;

#10 reset=0;

end

endmodule
```

Output:



Task 4:

Description:

In this 4th task, I basically make a 32-bit working PC with IM along with test bench for verifying the results. In PC, I took clk, reset and PC_out and when reset is zero then PC_out is incremented to 4 , else zero output and then in IM, I put PC_out output in address of IM and then Instruction in that address as operation of addi, add, sub and I made another module for connecting PC and IM by calling both modules and also truncate the instruction for indexing and in test bench, I verified the output simultaneously.

Code:

```
module program_counter( clk, reset, pc_out);

input clk, reset;

output reg [31:0] pc_out;

always @ (posedge clk or posedge reset)

begin

   if(reset)

     pc_out <= 32'h00000000;

   else

    pc_out <= pc_out + 32'd4;

end

endmodule

module instruction_memory(addr, instruction);

input [31:0] addr;

output reg [31:0] instruction;

reg [31:0] memory [0:15];

initial

begin

memory[0]  = 32'h00500093; // addi x1, x0, 5   (At byte addr 0x00)

memory[1]  = 32'h00a00113; // addi x2, x0, 10  (At byte addr 0x04)

memory[2]  = 32'h002081b3; // add  x3, x1, x2  (At byte addr 0x08)

memory[3]  = 32'h40208233; // sub  x4, x1, x2  (At byte addr 0x0C)

memory[4]  = 32'h00000000; // NOP  (At byte addr 0x10)

memory[5]  = 32'h00000000; // NOP
```

```
memory[6]  = 32'h00000000; // NOP

memory[7]  = 32'h00000000; // NOP

memory[8]  = 32'h00000000; // NOP

memory[9]  = 32'h00000000; // NOP

memory[10] = 32'h00000000; // NOP

memory[11] = 32'h00000000; // NOP

memory[12] = 32'h00000000; // NOP

memory[13] = 32'h00000000; // NOP

memory[14] = 32'h00000000; // NOP

memory[15] = 32'h00000000; // NOP

end

always @ (*)

begin

 instruction= memory[addr[5:2]];

end

endmodule

module top_rv32_fetch(clk, reset, pc_out, instruction);

input clk, reset;

output [31:0] pc_out;

output [31:0] instruction;

program_counter pc(.clk(clk), .reset(reset), .pc_out(pc_out));

instruction_memory im(.addr(pc_out), .instruction(instruction));

endmodule
```

```
module tb_program_counter();

reg clk, reset;

wire [31:0] pc_out;

wire [31:0] instruction;

top_rv32_fetch uut(.clk(clk),.reset(reset),.pc_out(pc_out),.instruction(instruction));

always begin

#5 clk= ~clk;

end

initial

begin

clk=0;

reset=0;

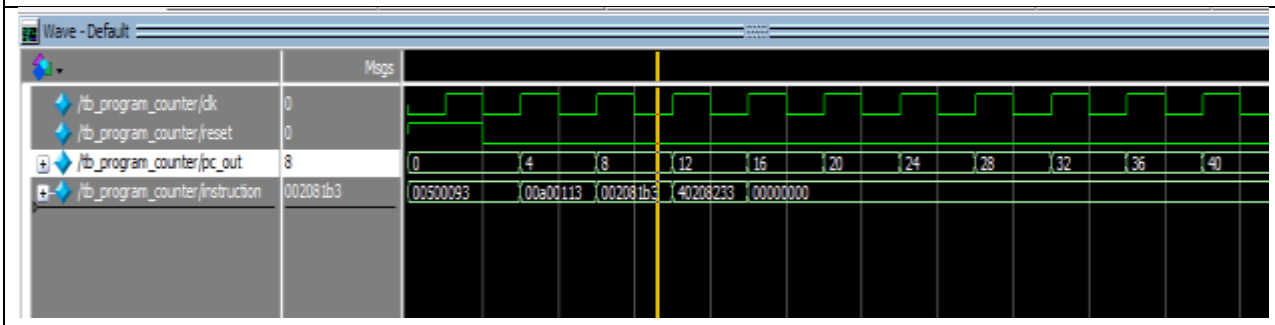
reset=1;

#10 reset=0;

end

endmodule
```

Output:



Task 5:

Description:

In this last task, I took register PC and enable to 1 and clock the signal and reset to 0 and output of register went to ADDER for PC increment to 4 by output and then the output from PC went to splitter for converting the hexadecimal digits by tructing the lower bits for indexing and then we set the addi, add and sub operations in ROM as IM and select the pin to 1 and then output is connected to probe by converting to hexadecimal.

Output:

